NCollection ERP Platform — Project Management Guide

Version: 2.0
Date: July 16, 2026
Classification: Internal — Project Management Reference
Prepared By: Architecture & Planning Team (Gemini, Claude, ChatGPT)
Purpose: The definitive guide for project management, timeline estimation, sprint planning, tooling strategy, CI/CD pipeline design, release management, risk analysis, and operational procedures for the NCollection ERP Platform.

Table of Contents

---

1. Project Timeline Overview

1.1 Total Project Estimate

Metric	Value
Total Phases	10
Total Tasks	96 atomic tasks
Team Size	3 remote developers + 3 AI agents
Estimated Duration	14–18 months (realistic range)
First Production Deployment	End of Phase 2 (~4 months from now)
Full Platform Completion	~16–18 months

Reason for the 14–18 month range: The project includes two experimental phases (AI Platform, Marketplace) and one enterprise-hardening phase that depend heavily on real-world feedback from early tenants.

Benefits of this timeline: Allows for iterative delivery — production deployment after Phase 2 generates revenue while later phases are developed.

Risks: Scope creep from client feedback, OCA module incompatibilities, team member unavailability.

Recommendation: Target the lower bound (14 months) but plan for 18 months in client communications.

1.2 Master Timeline

```
   ALREADY COMPLETED          CURRENT          FUTURE
   ________________________    __________     _________

 ┌────────────────────────────────┐
 | Docker, PostgreSQL,  |
 | White Label, SaaS   |  Month 1—3           Month 4—6
 | Foundation, OCA,    |  ┌─────────────────────────────┐
┌─────────────────────────────┐
 | MIS Builder, Demo,   |   | Phase 1:          |  | Phase 2: SaaS     |
 | Dashboard, Branding, |   | Customer Workspace |  | Automation        |
 | CI/CD Foundation     |   | ◄── CURRENT ──►    |  |                   |
 └────────────────────────────────┘  └─────────────────────────────┘
| Phase 3: ERP + UAE |
                              | (overlapping)     |
                              └─────────────────────────────┘

Month 7—8          Month 9—11          Month 12—14          Month 15—18
 ┌──────────────────────────┐   ┌──────────────────────────┐
┌──────────────────────────┐   ┌──────────────────────────┐
 | Phase 4:       |  | Phase 5: AI   |  | Phase 7: Mobile |  | Phase 9:      |
 | Dashboards      |  | Platform     |  |               |  | Marketplace    |
 |          |  |         |  | Phase 8: Platform |  |           |
 |          |  | Phase 6: Portal |  | Services       |  | Phase 10:      |
 |          |  | (overlapping)  |  |          |  | Enterprise Ready |
 └──────────────────────────┘   └──────────────────────────┘
└──────────────────────────┘   └──────────────────────────┘

★ FIRST PRODUCTION DEPLOY: End of Month 4 (after Phase 2 + Phase 3 UAE)
```

1.3 Milestone Calendar

Milestone	Target	Deliverable
M1: Infrastructure Hardened	Week 2	Docker hardening, CI enhancement, addon skeletons
M2: Tenant Routing Working	Week 4	Subdomain→database routing functional
M3: Customer Workspace MVP	Week 8	Branding + visibility + roles + dashboard
M4: Phase 1 Complete	Week 10–12	Full integration test passed
M5: SaaS Automation MVP	Week 16	Auto-provisioning + billing working
M6: UAE Localization Done	Week 18	VAT, CoA, Arabic, PDF invoices
M7: First Production Deploy	Week 18	Real tenants onboarded
M8: Dashboards Complete	Week 24	CEO + department dashboards
M9: AI Platform MVP	Week 32	Chat assistant + smart search
M10: Portal + Mobile MVP	Week 44	Portal redesigned, mobile beta

M11: Platform Services	Week 52	REST API, webhooks, monitoring
M12: Marketplace + Enterprise	Week 64–72	Full platform operational

---

2. Phase-by-Phase Estimation

2.1 Time Allocation Ratios

Each phase allocates time across three work stages. The ratios shift by phase complexity and novelty.

Phase	Analysis	Design	Implementation	Rationale
1. Customer Workspace	10%	25%	65%	Architecture decisions already made; focus is execution. Design effort on dashboard UX and visibility engine.
2. SaaS Automation	10%	15%	75%	Architecture is set; mostly pipeline engineering.
3. ERP + UAE	20%	15%	65%	UAE accounting standards require research. Implementation is configuration-heavy (XML data files).
4. Dashboards	10%	30%	60%	UX design iteration for dashboards; data query optimization.
5. AI Platform	25%	25%	50%	Most experimental phase. Significant analysis for LLM integration patterns and prompt engineering.
6. Customer Portal	10%	20%	70%	Odoo portal well-documented; mostly customization.
7. Mobile	20%	25%	55%	New technology stack requires framework evaluation and API design.
8. Platform Services	15%	25%	60%	API design is critical; implementation is straightforward once designed.
9. Marketplace	15%	25%	60%	Business model design (revenue sharing, moderation) needs careful thought.
10. Enterprise	20%	30%	50%	HA, scaling, compliance require extensive analysis and testing.

2.2 Detailed Phase Estimates

Phase	Tasks	Dev-Days	Calendar Weeks	Priority	Notes
1. Customer Workspace	16	57	8–12	P0	**CURRENT SPRINT**
2. SaaS Automation	10	40	6–8	P1	Starts after P1 stabilization
3. ERP + UAE	10	32	5–7	P1	Overlaps with Phase 2 (DEV-2/3)
4. Dashboards	4	17	3–4	P2	Relatively lightweight
5. AI Platform	6	27	4–6	P3	Experimental — budget 25% for spikes
6. Customer Portal	5	22	3–5	P3	Depends on Phase 2 billing
7. Mobile	6	30	6–8	P3	New stack; longest single-dev load
8. Platform Services	7	32	5–7	P3	REST API is the largest task
9. Marketplace	6	31	5–7	P3	Revenue sharing model needs business input
10. Enterprise	8	45	8–10	P3	HA and scaling are infrastructure-

				intensive
TOTAL	**78**	**333**	**53–74**	

333 dev-days ÷ 3 developers ÷ 5 days/week = ~22 work-weeks per developer
With reviews, debugging, meetings, iteration, and holidays: **~53–74 calendar weeks (12–18 months)**

2.3 Developer Workload Distribution (Phase 1 — Current Sprint)

Developer	Tasks	Total Days	Critical Path?
DEV-1	P1-T01, T02, T03, T05, T13, T16	19	Yes — T01 blocks everything
DEV-2	P1-T04, T06, T07, T08	15	Yes — T04 blocks T06
DEV-3	P1-T09, T10, T11, T12, T14, T15	23	⚠ Heaviest load

Recommendation: If DEV-3 is bottlenecked, shift P1-T15 (Email Template Branding) to DEV-2 (who has capacity in Week 4–5).

2.4 Critical Path Analysis

The critical path (longest sequential chain) for Phase 1:

```
P1-T01 → P1-T03 → P1-T04 → P1-T06 → P1-T16
 4d      1d      5d      5d      3d    = 18 working days minimum
```

Implication: Even with infinite developers, Phase 1 cannot complete faster than **~4 calendar weeks** of pure development time (plus buffer).

---

3. Sprint Planning & Weekly Schedule

3.1 Sprint Structure

Sprint Duration: 2 weeks (10 working days)

Reason: 2-week sprints balance delivery frequency with meaningful progress. 1-week sprints create too much ceremony overhead for a 3-person team; 3-week sprints delay feedback loops.

Benefits: Regular demo cadence, predictable velocity tracking, manageable sprint planning scope.

Risks: Incomplete tasks may spill over. Mitigation: strict task sizing (max 5 days per task).

Alternative: Kanban flow without fixed sprints (rejected: harder to track velocity and estimate future phases).

Recommendation: 2-week sprints with a structured ceremony calendar.

3.2 Sprint Ceremony Calendar

Day	Event	Duration	Attendees	Purpose
Sprint Day 1 (Monday)	Sprint Planning	1 hour	All DEVs + Omar	Select issues from Ready column; assign and commit
Daily (async)	Daily Standup	5 min text	All DEVs	Post in Discord: done yesterday, doing today, blockers

Sprint Day 5 (Friday)	Mid-Sprint Check	15 min (voice)	All DEVs	Review progress; surface blockers early
Sprint Day 10 (Friday)	Sprint Review/Demo	30 min (video)	All DEVs + Omar + Client (optional)	Demo completed work; get feedback
Sprint Day 10 (Friday)	Sprint Retrospective	15 min	All DEVs	What went well, what didn't, improvements

3.3 Weekly Developer Schedule Template

```
Monday:
  ├── Sprint Planning (if Sprint Day 1) or continue current tasks
  ├── Code implementation (focus time — 4+ hours uninterrupted)
  └── Post daily standup in Discord before EOD

Tuesday–Thursday:
  ├── Code implementation (focus time — 6+ hours)
  ├── Code reviews (allocate 30–60 min)
  ├── Post daily standup in Discord
  └── AI agent consultation as needed (Claude for implementation, ChatGPT for design)

Friday:
  ├── Code implementation + PR cleanup
  ├── Code reviews (clear all pending PRs)
  ├── Mid-Sprint Check (if Sprint Day 5) or Sprint Review (if Sprint Day 10)
  ├── Documentation updates
  └── Post daily standup in Discord
```

3.4 Sprint Goal Template

Each sprint must have a clear, measurable goal:

```
## Sprint [N] Goal
**Dates**: July 16 – July 30, 2026
**Phase**: 1 — Customer Workspace
**Goal**: Complete tenant routing, module visibility, and branding to the point where
      two test tenants with different plans see different module sets.

### Committed Tasks
| Task ID | Task Name | Assigned | Status |
|---------|-----------|----------|--------|
| P1-T01  | Docker Hardening | DEV-1 |      In Progress |
| P1-T03  | Addon Skeletons | DEV-1 |      In Progress |
| P1-T07  | Tenant Roles | DEV-2 |      Ready |
| P1-T09  | Branding Completion | DEV-3 |      Ready |

### Success Criteria
- [ ] `docker compose up` starts with Nginx, multi-tenant odoo.conf
- [ ] Two test databases accessible via different subdomains
- [ ] All NCollection role groups defined and importable
- [ ] Zero Odoo references visible in the UI
```

4. GitHub Projects Strategy

4.1 Project Board Setup

Reason: GitHub Projects (v2) provides integrated Kanban boards, timeline views, and custom fields — sufficient for a 3-dev team without the overhead of Jira or Linear.

Benefits: Zero context switching (issues, PRs, and board are all GitHub); free for private repos; built-in automation.

Risks: Limited reporting compared to enterprise PM tools. Mitigation: use GitHub Insights + manual sprint velocity tracking.

Alternative: Jira (rejected: too heavyweight for 3 devs; adds $7.75/user/month; requires context switching); Linear (viable alternative, but GitHub native is preferred for zero-friction).

Recommendation: Use GitHub Projects v2 with the configuration below.

4.2 Board Configuration

Columns (Status Field):

Column	Icon	Description
Backlog		All future tasks not yet ready for development
Ready		Dependencies met, sized, ready to be picked up
In Progress		Currently being worked on (max 2 per developer)
In Review		PR submitted, awaiting review
Done		Merged to develop and verified
Blocked		Cannot proceed due to external dependency

Custom Fields:

Field	Type	Purpose
Phase	Single Select	Phase 1–10
Developer	Single Select	DEV-1, DEV-2, DEV-3
Priority	Single Select	P0, P1, P2, P3
Estimate (days)	Number	Task size in developer-days
Sprint	Iteration	2-week sprint assignment
Complexity	Single Select	Low, Medium, High, Very High

Views:

View	Type	Purpose
Sprint Board	Board (by Status)	Current sprint work — default view
All Tasks Table	Table	Complete backlog with all fields visible, sortable
Timeline	Roadmap	Gantt-style phase view (available in GitHub Projects v2)
By Developer	Board (by Developer)	See each developer's workload

Automation Rules (GitHub Projects Workflows):

Trigger	Action
PR linked to issue is opened	Move issue to "In Review"

PR linked to issue is merged	Move issue to "Done"
Issue labeled status:blocked	Move issue to "Blocked"
Issue assigned to someone	Move issue to "In Progress"

---

5. GitHub Issues Hierarchy & Labels

5.1 Issue Hierarchy

```
Milestone (Phase) ──▶ Epic (Feature Area) ──▶ Issue (Atomic Task)
                                          │
                                          └── Sub-tasks (Checkboxes within Issue body)
```

Example:

```
Milestone: Phase 1 — Customer Workspace
  └── Epic: Module Visibility [label: epic:module-visibility]
        ├── Issue: P1-T04 — Tenant Model Enhancements
        ├── Issue: P1-T06 — Module Visibility Engine
        └── Issue: P1-T08 — Apps & Settings Menu Stripping
```

5.2 Label Taxonomy

Phase Labels (Blue shades):

Label	Color
phase:1-workspace	#0052CC
phase:2-saas	#0065FF
phase:3-erp	#0078D7
phase:4-dashboards	#008CBA
phase:5-ai	#00A0DC
phase:6-portal	#00B4D8
phase:7-mobile	#48CAE4
phase:8-platform	#90E0EF
phase:9-marketplace	#ADE8F4
phase:10-enterprise	#CAF0F8

Developer Labels (Green shades):

Label	Color
dev:DEV-1	#2EA043
dev:DEV-2	#3FB950
dev:DEV-3	#56D364

Priority Labels (Red shades):

Label	Color
priority:P0-immediate	#D73A49
priority:P1-high	#E36209
priority:P2-medium	#FBCA04

priority:P3-future	#C5DEF5

Type Labels (Purple shades):

Label	Color
type:feature	#7057FF
type:bug	#D73A49
type:infra	#0075CA
type:docs	#1D76DB
type:spike	#BFD4F2
type:chore	#CCCCCC
type:security	#B60205

Status Labels (Yellow shades):

Label	Color
status:blocked	#E4E669
status:needs-design	#FEF2C0
status:needs-oca-check	#FBCA04

Layer Labels (Teal):

Label	Color
layer:platform	#006B75
layer:erp	#0E8A16

5.3 Issue Template

```markdown
## [P1-T04] Tenant & Subscription Model Enhancements

**Phase**: 1 — Customer Workspace
**Layer**: Platform
**Assigned**: DEV-2
**Priority**: P0
**Estimated Days**: 5
**Complexity**: High
**Dependencies**: P1-T03 (Addon Skeleton Finalization)

### Context
The `ncollection.tenant`, `ncollection.subscription`, and `ncollection.subscription.plan`
models already exist with basic fields and ORM integration. This task enhances them with
business logic required for the Customer Workspace phase.

### Description
[Detailed description from Deliverable 1]

### OCA Check Required
- [ ] Searched OCA repositories for existing solution
- [ ] OCA module found: [name] / Not applicable
- [ ] Decision: Use OCA / Build custom (with justification)

### Acceptance Criteria
- [ ] `module_ids` field on subscription plan works correctly
- [ ] State transitions validate current state before transition
- [ ] `days_remaining` computed field displays correctly
- [ ] Mail notifications fire on status changes
- [ ] All `_check_` constraints raise proper ValidationError

### Technical Notes
- Do NOT use Many2many to `ir.module.module` (cross-DB complication)
- Use comma-separated text field for module technical names

### Definition of Done
- [ ] Code passes CI (flake8 + pylint-odoo)
- [ ] Unit tests pass for state transitions
- [ ] PR approved by at least 1 reviewer
- [ ] Merged to `develop`
- [ ] Documentation updated (model fields, transitions)
```

5.4 Milestone Setup

Milestone	Due Date	Issues
Phase 1: Customer Workspace	Week 12	16
Phase 2: SaaS Automation	Week 18	10
Phase 3: ERP + UAE	Week 20	10
Phase 4: Dashboards	Week 24	4
Phase 5: AI Platform	Week 32	6
Phase 6: Customer Portal	Week 36	5

Phase 7: Mobile	Week 44	6
Phase 8: Platform Services	Week 52	7
Phase 9: Marketplace	Week 60	6
Phase 10: Enterprise Readiness	Week 72	8

---

6. Branch Strategy & Git Flow

6.1 Branch Architecture

```
main ◄──────────────────────────────── Production releases (tagged: v1.0.0, v1.1.0, ...)
  │
  └──── develop ◄──────────────── Integration branch (all feature PRs merge here)
          │
          ├──── feature/P1-T01-docker-hardening     ◄──── Feature branches
          ├──── feature/P1-T06-module-visibility
          ├──── feature/P1-T09-branding-completion
          │
          ├──── hotfix/csrf-login-fix          ◄──── Emergency fixes
          │
          └──── release/v1.0.0                 ◄──── Release preparation
```

Reason: Git Flow with main/develop separation provides clear production/integration boundaries. Feature branches keep developers isolated from each other's work-in-progress.

Benefits: Clean commit history; predictable deployment pipeline; clear rollback points.

Risks: Merge conflicts on long-lived feature branches. Mitigation: rebase feature branches on develop daily; keep PRs small (< 400 LOC).

Alternative: Trunk-based development (rejected: requires robust CI/CD and feature flags — too much infra for Phase 1).

Recommendation: Use Git Flow with the naming convention and rules below.

6.2 Branch Naming Convention

Type	Pattern	Example
Feature	feature/P{phase}-T{id}-{short-desc}	feature/P1-T06-module-visibility
Hotfix	hotfix/{desc}	hotfix/login-csrf-fix
Release	release/v{semver}	release/v1.0.0

6.3 Branch Protection Rules

main branch:

- Require pull request (no direct push)
- Require 1 approval
- Require CI to pass
- Require branch up-to-date before merge
- No force push
- No deletions

develop branch:

- Require pull request
- Require 1 approval
- Require CI to pass
- ☐ Branch up-to-date NOT required (to avoid rebase hell)

6.4 Merge Strategy

Merge Type	When	Why
Squash merge	Feature → develop	Clean history; one commit per feature
Merge commit	develop → main (release)	Preserve full history for audit
Rebase	Feature branch update from develop	Keep feature branch linear

---

7. Docker Strategy

7.1 Is Docker Necessary?

Verdict: Docker is a **non-negotiable requirement** for this project.

Reason: NCollection is a multi-tenant SaaS platform. Docker provides the environment isolation, reproducibility, and operational tooling required to serve multiple tenants reliably. The Docker infrastructure is already completed and running.

Benefits:

Benefit	Explanation
Environment Parity	Identical environments across dev/staging/production. "Works on my machine" eliminated.
New Dev Onboarding	docker compose up — everything runs in minutes, not hours.
Multi-Tenant Safety	Process-level isolation. PgBouncer, Redis, Nginx are separate containers.
Scaling	Add Odoo worker containers behind load balancer. Docker Swarm/K8s for orchestration.
Rollback	Pin image versions. Rollback = point to previous tag.
Zero-Downtime Deploys	Rolling updates or blue-green deployment with Docker.
Resource Control	CPU/memory limits per container prevent runaway processes.
CI/CD Integration	Build Docker images in CI; deploy by pulling latest image.

Risks:

Risk	Mitigation
Docker learning curve	1-day Docker workshop; provide cheat sheet
Volume permission issues (common with Odoo)	Set proper UID mapping; use user: root in dev only
Docker Desktop licensing on Mac/Windows	Use Docker Engine directly (Linux), Colima/OrbStack on Mac
Container orchestration complexity at scale	Start with Docker Compose; move to Swarm/K8s only when needed

Alternatives:

Alternative	Why Rejected
Bare-metal installation	No environment parity; complex onboarding; no resource isolation; manual scaling
Virtual machines (VMware, Vagrant)	Heavier than containers; slower startup; more resource-intensive
Platform-as-a-Service (Heroku, etc.)	Insufficient control for multi-tenant Odoo; expensive at scale

Recommendation: Continue with Docker. The existing setup is the foundation — extend it per the Phase 1 task P1-T01 (Docker Environment Hardening).

7.2 Docker Environments

Environment	File	Purpose
Base	docker-compose.yml	Core services (db, odoo) — shared config
Development	docker-compose.dev.yml	Override: pgadmin, single worker, debug mode, source mount
Production	docker-compose.prod.yml	Override: Nginx, Redis, PgBouncer, resource limits, no pgadmin

Usage:

```
# Development
docker compose -f docker-compose.yml -f docker-compose.dev.yml up

# Production
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

7.3 Custom Production Dockerfile

```
FROM odoo:19

# Copy custom addons into the image (baked in for production)
COPY ./custom_addons /mnt/extra-addons

# Copy production odoo.conf
COPY ./config/odoo.conf /etc/odoo/odoo.conf

# Install Python dependencies for custom addons (if any)
# COPY requirements.txt /tmp/requirements.txt
# RUN pip install --no-cache-dir -r /tmp/requirements.txt

EXPOSE 8069 8072
```

Reason: Baking addons into the image ensures production deployments are reproducible and don't depend on volume mounts.

---

8. CI/CD Pipeline

8.1 Current State

The existing CI pipeline (.github/workflows/ci.yml) runs flake8 on PRs to develop and main.

8.2 Target Pipeline Architecture

```
┌──────────────────────────────────────── Pull Request to develop
────────────────────────────────────────┐
│                                   │
│  ┌──── Stage 1: Lint ────┐  ┌──── Stage 2: Test ────┐  ┌── Stage 3: Build ┐
│  │ flake8          │  │ Start PostgreSQL   │  │ docker build    │
│  │ pylint-odoo       │  │ Install modules    │  │ Start containers │
│  │ XML validation      │  │ Run --test-enable    │  │ HTTP smoke test  │
│  └────────────────────────────────────────┘
└────────────────────────────────────────┘  └──────────────────────────────┘
│                                   │
│  All pass →   PR can be merged                     │
└────────────────────────────────────────────────────────────────────────────
─────────────────────────────────────────┘

┌──────────────────────────────────────── Push to develop
───────────────────────────────────────────────────┐
│                                   │
│  ┌──── Build ─────┐  ┌──── Deploy Staging ────┐  ┌──── Smoke Test ────┐   │
│  │ docker build │  │ SSH → staging VPS   │  │ curl /web/login  │   │
│  │ docker push  │  │ docker compose pull  │  │ Verify HTTP 200  │   │
│  │ to registry  │  │ docker compose up -d │  │ Notify Discord   │   │
│  └──────────────────────────┘  └────────────────────────────────────┘
└──────────────────────────────────┘  │
└────────────────────────────────────────────────────────────────────────────
─────────────────────────────────────────┘

┌──────────────────────────────────────── Push to main (release)
────────────────────────────────────────┐
│                                   │
│  ┌──── Tag ─────┐  ┌──── Build ────┐  ┌──── Deploy Production ────┐     │
│  │ git tag   │  │ docker build │  │ MANUAL APPROVAL REQUIRED │     │
│  │ v1.x.x   │  │ docker push  │  │ SSH → prod VPS      │     │
│  └────────────────┘  └────────────────────────┘  │ docker compose up -d    │     │
│             │ Run DB migrations     │     │
│             │ Smoke test        │     │
│             │ Notify team        │     │
│             └──────────────────────────────────────┘     │
└────────────────────────────────────────────────────────────────────────────
─────────────────────────────────────────┘
```

8.3 CI Workflow File (Target)

```yaml
name: NCollection ERP CI

on:
  pull_request:
    branches: [develop, main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: |
          pip install flake8 pylint-odoo
          flake8 custom_addons/
          # pylint-odoo will be added in P1-T02

  test:
    runs-on: ubuntu-latest
    needs: lint
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: odoo
          POSTGRES_PASSWORD: odoo
          POSTGRES_DB: test_db
        ports: ['5432:5432']
    steps:
      - uses: actions/checkout@v4
      # Odoo test runner steps (implemented in P1-T02)

  build:
    runs-on: ubuntu-latest
    needs: lint
    steps:
      - uses: actions/checkout@v4
      - run: docker compose build
      - run: |
          docker compose up -d
          sleep 30
          curl -f http://localhost:8069/web/login || exit 1
          docker compose down
```

Reason: Multi-stage pipeline catches progressively deeper issues. Lint catches syntax; tests catch logic; build catches runtime.

Benefits: High confidence in merge quality; automated regression detection.

Risks: CI time may reach 5–8 minutes. Mitigation: run lint and build in parallel; tests sequentially.

Recommendation: Implement incrementally — lint first (done), then build smoke test (P1-T02), then full test runner (P1-T02).

---

9. Code Review Process

9.1 Rules

Rule	Description	Reason
No direct push	All code goes through PRs	Ensures review and CI validation
1 approval minimum	At least 1 other DEV must approve	Four-eyes principle; knowledge sharing
No self-merge	Author cannot merge their own PR	Exception: critical hotfixes with team notification
Max PR size	400 lines of changed code (excluding auto-generated files)	Large PRs receive superficial reviews
24-hour turnaround	Reviewer responds within 1 business day	Prevents PR backlog
Squash merge	Feature branches use squash merge to develop	Clean commit history
CI must pass	Green CI is a merge prerequisite	No broken code in integration branch

9.2 PR Template

```
## [P1-T06] Module Visibility Engine

**Task ID**: P1-T06
**Phase**: 1 — Customer Workspace
**Layer**: Platform / ERP (specify)

### What Changed
- Overrode `ir.ui.menu._visible_menu_ids()` in `ncollection_core`
- Added `ncollection.workspace.config` model for per-tenant module list
- Added XML data file with default workspace config

### Why
Tenants must only see modules included in their subscription plan.
The visibility engine reads allowed modules from a local config record
(synced during provisioning) to avoid cross-database queries.

### OCA Check
- [x] Checked OCA for existing module visibility solutions
- Result: No suitable OCA module found for subscription-based menu hiding

### Testing
- [ ] Starter plan tenant sees: CRM, Sales, Invoicing
- [ ] Enterprise plan tenant sees: All modules
- [ ] Direct URL access to hidden module returns appropriate error

### Screenshots
[If UI changes, include before/after screenshots]

### Checklist
- [ ] Code follows Odoo conventions
- [ ] No Odoo core files modified
- [ ] Two-layer separation respected
- [ ] Unit tests added/updated
- [ ] Documentation updated
- [ ] Upgrade compatibility maintained
```

9.3 Review Checklist (for Reviewers)

- [ ] Does the code follow the two-layer architecture? (Platform vs. ERP)
- [ ] Are there any Odoo core modifications? (MUST be zero)
- [ ] Was OCA checked before building this feature?
- [ ] Are _inherit overrides documented with reason?
- [ ] Are there unit tests for new model methods?
- [ ] Does this work correctly in a multi-tenant context?
- [ ] Are there any security concerns (XSS, SQL injection, access bypass)?
- [ ] Is the code upgrade-compatible with future Odoo versions?
- [ ] Is the PR size reasonable (< 400 LOC)?

---

10. Release Process

10.1 Release Cadence

Reason: Monthly releases balance feature delivery with stability. Weekly releases are too frequent for a 3-person team; quarterly releases delay value delivery.

Recommendation: Monthly releases after Phase 1; bi-weekly during Phase 1 (faster iteration on the foundation).

10.2 Release Workflow

```
1. Sprint ends → All committed tasks merged to `develop`
2. Release Manager (DEV-1) creates `release/v1.x.0` branch from `develop`
3. Release branch: bug fixes only (no new features)
4. QA testing on staging environment
5. Release notes written (changelog, breaking changes, migration steps)
6. `release/v1.x.0` merged to `main` via PR (requires 2 approvals)
7. Tag `v1.x.0` on `main`
8. Deploy to production (manual trigger)
9. Post-deployment smoke tests
10. Announce release to team and client
11. Merge `main` back to `develop` (to capture any release bug fixes)
```

10.3 Semantic Versioning

Version	When
v0.1.0	Phase 1 complete (Customer Workspace MVP)
v0.2.0	Phase 2 complete (SaaS Automation)
v0.3.0	Phase 3 complete (UAE Localization)
v1.0.0	First production deployment (after Phase 2 + 3)
v1.1.0	Phase 4 complete
v2.0.0	Major milestone (AI Platform or Mobile)

---

11. Production Deployment

11.1 Deployment Steps

```
# 1. SSH into production server
ssh deploy@prod.ncollectionerp.com

# 2. Pull latest images
cd /opt/ncollection
docker compose -f docker-compose.yml -f docker-compose.prod.yml pull

# 3. Pre-deployment backup
./scripts/backup-all-tenants.sh

# 4. Apply database migrations (if any)
docker compose exec odoo odoo-bin -u ncollection_subscription -d admin_db --stop-after-init

# 5. Rolling restart
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --no-deps odoo

# 6. Verify health
curl -f https://admin.ncollectionerp.com/web/login || echo "DEPLOYMENT FAILED"

# 7. Run smoke tests
./scripts/smoke-test.sh

# 8. Notify team
curl -X POST $DISCORD_WEBHOOK -d '{"content": "   v1.x.0 deployed to production"}'
```

11.2 Rollback Procedure

```
# 1. Revert to previous image tag
docker compose -f docker-compose.yml -f docker-compose.prod.yml \
  up -d --no-deps odoo  # (with previous image tag in .env)

# 2. If DB migration was applied: restore from pre-deployment backup
./scripts/restore-backup.sh admin_db backup_20260716_pre_deploy.dump

# 3. Verify
curl -f https://admin.ncollectionerp.com/web/login

# 4. Notify team
curl -X POST $DISCORD_WEBHOOK -d '{"content": "⚠ Rollback to v1.x-1.0 completed"}'
```

11.3 Zero-Downtime Deployment (Phase 10)

Current approach: Brief downtime during restart (~30 seconds).

Target approach (Phase 10): Blue-green deployment with health check-based switchover.

---

12. Backup Strategy

12.1 Backup Levels

Level	What	How	Frequency	Retention

Database	All tenant PostgreSQL databases	pg_dump --format=custom per database	Daily (02:00 UTC)	7 daily, 4 weekly, 12 monthly
Filestore	Odoo attachments per tenant	tar of /data/odoo/filestore/<db>/	Daily (03:00 UTC)	7 daily, 4 weekly, 12 monthly
Configuration	Docker configs, Nginx, odoo.conf	Git repository (already versioned)	On every commit	Infinite (Git history)
Pre-deployment	Full snapshot before any deployment	pg_dump all DBs + filestore tar	Before every deploy	Keep 5 most recent

12.2 Storage

Reason: Cloud object storage (S3 or Backblaze B2) provides durable, geographically redundant backup storage at low cost.

Provider	Cost	Durability	Used For
Backblaze B2	$0.005/GB/month	99.999999999%	Daily backups (primary)
Local disk	$0 (included)	Single disk	Pre-deployment snapshots (secondary)

Benefits: Off-site storage protects against server failure; retention policy manages storage costs.

Risks: Backup restoration must be tested regularly. Untested backups are not backups.

Recommendation: Schedule monthly backup restoration drills. Restore a random tenant backup to a test database and verify data integrity.

12.3 Backup Verification

Check	Frequency	How
Backup job completed	Daily (automated)	Check ncollection.backup records; alert on failures
Backup file exists in cloud	Weekly (automated)	Script to list recent backups in B2 bucket
Backup restore test	Monthly (manual)	Restore random tenant to test DB; verify data
Full disaster recovery drill	Quarterly	Simulate complete server loss; rebuild from backups

---

13. Collaboration Stack

13.1 Recommended Tools

Category	Tool	Reason	Cost
Version Control	GitHub (Private)	Already in use; native Issues, PRs, Actions integration	Free
Project Management	GitHub Projects v2	Kanban + timeline; zero context switching from code	Free
CI/CD	GitHub Actions	Already configured; 2000 min/month free for private repos	Free
Communication	Discord	Real-time chat; voice channels for syncs; free for teams	Free
Video Calls	Google Meet / Discord Voice	Weekly syncs; sprint ceremonies	Free
	Markdown in docs/ +		

Documentation	GitHub Wiki	Version-controlled; co-located with code	Free
Design/Wireframes	Figma	Dashboard/portal/mobile UI design; free for 3 editors	Free
Database Admin	pgAdmin (Docker)	Already in Docker Compose; development only	Free
API Testing	Bruno (Git-friendly) or Postman	REST API testing (Phase 8)	Free
AI Pair Programming	Claude (Antigravity)	Implementation partner; follows project rules	Included
AI Architecture	ChatGPT	Solution design; OCA evaluation	Subscription
AI Planning	Gemini (Antigravity)	Architecture review; planning documents	Included

Total Monthly Cost for Collaboration Tools: **$0** (everything is free tier or already included).

Alternative: Slack + Jira + Confluence ($20+/user/month) — rejected as unnecessarily expensive and complex for a 3-person team.

Recommendation: Use the free stack above. Re-evaluate when team grows beyond 5 developers.

---

14. Risk Analysis

14.1 Risk Register

#	Risk	Category	Probability	Impact	Risk Score	Mitigation	Owner
1	**OCA module incompatibility with Odoo 19**	Technical	Medium	High	High	Check branch availability before planning; test in dev before committing; maintain compatibility patches	DEV-1
2	**Multi-tenant routing complexity**	Technical	Low	Critical	High	POC in Week 1; extensive testing with 3+ tenant DBs; fallback to URL-path routing if subdomains fail	DEV-1
3	**Cross-tenant data leakage**	Security	Low	Critical	Critical	Database-per-tenant isolation; automated security tests; penetration testing before production	DEV-1
4	**Developer ramp-up on Odoo internals**	Team	Medium	Medium	Medium	Pair programming with AI agents; Odoo 19 internal documentation; allocate 1 week for learning	All
5	**Scope creep from client feedback**	Business	High	Medium	High	Strict phase-based development; change requests go to backlog; no mid-sprint scope changes	Omar
6	**Key developer unavailability**	Team	Medium	High	High	Cross-train on critical systems; document all decisions; AI agents can cover implementation gaps	Omar
7	**Odoo 20 release mid-project**	Technical	High	Medium	Medium	Ignore Odoo 20 until after Phase 3; plan migration as separate	DEV-1

						project; ensure Rule 4 compliance	
8	**Payment gateway integration complexity**	Technical	Medium	Medium	Medium	Start with Stripe (best documented); add PayTabs/Tap after Stripe works; check OCA modules first	DEV-1
9	**Mobile development timeline overrun**	Technical	High	Medium	High	Choose framework early (React Native vs Flutter); start with PWA as fallback; limit initial screens	DEV-3
10	**LLM API pricing/availability changes**	External	Low	Low	Low	Abstract LLM provider behind gateway; support multiple providers; budget for API costs	DEV-1

14.2 Risk Response Strategy

Score	Response
Critical	Immediate action required; dedicated sprint to address; escalate to Omar
High	Active mitigation plan; monitor weekly; contingency plan documented
Medium	Monitor during sprint reviews; address if probability increases
Low	Accept; review quarterly

14.3 Scenario-Based Timeline Analysis

Scenario	Duration	Key Assumptions
Optimistic	12 months	All DEVs full-time, OCA modules compatible, no major blockers, client stable requirements
Realistic	14–16 months	1–2 week buffer per phase, some OCA issues, occasional DEV unavailability, minor scope adjustments
Pessimistic	18–20 months	Major OCA incompatibility, developer turnover, significant scope creep, Odoo 20 forced migration

Recommendation: Communicate the realistic timeline (14–16 months) to the client. Use the optimistic timeline as an internal stretch goal.

---

15. Budget & Infrastructure Costs

15.1 Monthly Infrastructure Costs

Item	Provider	Cost/Month	When Needed
VPS (Staging)	Hetzner CX32	~€16	Now
VPS (Production)	Hetzner CX42	~€27	Phase 2+
Domain	Any registrar	~$1/month	Now
SSL Certificates	Let's Encrypt	Free	Phase 1
Backup Storage	Backblaze B2	~$5	Phase 2
Email (Transactional)	Mailgun	~$15	Phase 2
GitHub	Free / Team	$0–$4/user	Now (free)
LLM API	OpenAI / Anthropic	~$50–200	Phase 5
Firebase FCM	Google	Free	Phase 7
Monitoring	Self-hosted Prometheus/Grafana	Free	Phase 8

Period	Estimated Monthly Cost
Phases 1–3 (now–Month 6)	**~$60/month**
Phases 4–6 (Month 7–11)	**~$150/month**
Phases 7–10 (Month 12–18)	**~$200–400/month**
Post-launch at scale (100+ tenants)	**~$500–1000/month**

15.2 Cost Scaling Model

```
Tenants:   1    10    50    100   500   1000
VPS:       €27  €27   €54   €108  €270  €540+
DB Size:   1GB  5GB   25GB  50GB  250GB 500GB
Backups:   $1   $5    $25   $50   $250  $500
Total/mo:  ~$60 ~$100 ~$250 ~$500 ~$1K  ~$2K
```

Break-even analysis: At an average subscription price of $50/month, the platform becomes profitable at ~10 tenants (covering infrastructure costs) and significantly profitable at 100+ tenants.

Document End
This Project Management Guide is a living document. Update after each sprint retrospective to reflect actual velocity, process improvements, and risk status changes.